ADR4AI: LLM-Based Architectural Pattern Selection and ADR Generation for AI-Enabled Systems

Anonymous Authors

Working draft: Tasks 1.1–1.9, 2.1–2.5, partial 4.1, 4.2, partial 4.3, 4.4, partial 5.1–5.4, 6.1–6.4, and partial 7.1–7.2. The Introduction gap and approach overview have been drafted for this review. The partial Introduction motivation and problem statement have been approved; explicit TBDs mark pending expert review, and Tasks 7.3–7.4 remain unfinished. The AI-system pattern positioning subsection has been approved for this draft. The ADR-generation related work subsection has been approved for this draft. The LLM-assisted architecture related work subsection has been approved for this draft. The Background subsection has been approved for this draft. The partial conclusion-validity subsection has been approved for this draft. The partial external-validity subsection has been approved for this draft. The partial internal-validity subsection has been approved for this draft. The partial construct-validity subsection has been approved for this draft. The research implications and future work have been approved for this draft. The partial practitioner implications have been approved for this draft. The comparative Discussion subsection has been approved for this draft. The partial Discussion opening has been approved for this draft; Phase 3 remains postponed. The expert-result placeholders have been approved for this draft. The ADR-generation Results subsection has been approved for this draft. The RQ2 Results subsection has been approved for this draft. The RQ1 Results subsection has been approved for this draft. Core computational counts have been reconciled and approved for this draft. The expert-analysis protocol has been approved for this draft. The proposed governance procedures have been approved for this draft; the required permissions and safeguards remain pending confirmation. This document is not a complete paper or a submission-ready manuscript. **EXPERT RESULTS UNRESOLVED—TBD**.

I. Introduction

Translating requirements into architectural decisions requires more than identifying components that implement requested functionality. Architects must interpret the operational context, consider quality attributes, and justify choices whose consequences affect subsequent system evolution [1], [2]. For AI-enabled systems, this task also involves concerns spanning data preparation, model training, deployment, and monitoring [3], [4]. A user story and its acceptance criteria may describe expected behavior without specifying the data and model lifecycle assumptions needed to select an architectural solution. Decision support must therefore distinguish what the requirements establish from what would require additional information.

Architectural patterns provide a vocabulary of reusable solutions to recurring problems. Catalogs for AI-based systems organize this knowledge and make candidate solutions accessible [4], [5]. However, the existence of a relevant pattern does not establish its applicability to a particular requirement. Selection requires examining the pattern's problem, context, and consequences against the available requirement evidence, rather than relying on shared terminology alone. Reuse also needs an explicit rationale: an Architectural Decision Record (ADR) can document the context, proposed decision, and consequences so that the choice remains inspectable and can be revisited [6]. Pattern identification and decision documentation are thus connected but distinct architectural activities.

Large language models (LLMs) offer opportunities to assist these activities by interpreting natural-language inputs and drafting architectural artifacts [7], [8]. Nevertheless, fluent output is not evidence of a sound architectural decision. Empirical research on LLM architectural knowledge identifies limitations in precision and the recovery of design rationale [9]. A plausible recommendation may depend on assumptions absent from the requirements, while a well-structured ADR may still contain unsupported reasoning. Neither a model-generated applicability judgment nor successful document generation should therefore be treated as validation of architectural correctness.

The problem motivating ADR4AI is how to support the transition from industrial requirements to candidate AI-system patterns and inspectable ADR drafts while preserving the evidence behind each recommendation. Such support should expose the requirement passages used to justify applicability, distinguish insufficient information from a supported decision, and retain links between requirements, selected patterns, and generated records. Expert review is needed to assess whether the recommendations and their documented rationale are adequate for the stated requirements. The intended role is traceable decision support for architects, not autonomous acceptance of architectural decisions.

TBD—Expert review of pattern recommendations and ADR drafts pending Phase 3 execution and consolidation. The assessment of recommendation adequacy and documented rationale remains pending; no expert-validation outcome is claimed in this Introduction.

The literature reviewed in Section II examines LLM-assisted

architectural design, ADR decision generation, and AI-pattern solution synthesis [7], [8], [10]. These lines of work motivate investigating their intersection: requirement-grounded selection from an AI-pattern catalog linked to decision documentation and comparison with retrieval-based candidates. ADR4AI operationalizes this investigation through the pipeline in Figure 1. It assesses each of 70 catalog patterns against each of 12 industrial user stories and their acceptance criteria from 4 systems, yielding an exhaustive census of 840 story–pattern pairs within this bounded sample. The LLM classifies each pair as applicable, not applicable, or insufficient information, requiring requirement references for positive judgments. Each pair classified as applicable supplies the requirement context, pattern, and applicability evidence for a traceable ADR draft with Proposed status. A separate dense-retrieval and reranking baseline supplies a final Top-5 candidate set for same-story overlap comparison; this is an output-set comparison, not a test of architectural correctness or superiority. The prepared expert-assessment protocol complements these computational stages by asking practitioners familiar with the respective systems to assess selected patterns and their ADRs, including decision adequacy, grounding, consequences, perceived usefulness, and pattern applicability. **TBD—Expert assessment execution, consolidated responses, and resulting validation evidence pending Phase 3; this stage is planned, not reported as completed.**

II. Background and Related Work

A. Background

Architectural Design Decisions and ADRs. Software architecture is widely characterized as the composition of fundamental design decisions that shape a system's structure, behavior, and quality attributes [1], [2]. Capturing the context, trade-offs, and rationale behind these choices is critical for system evolution, knowledge transfer, and preventing architectural drift. Architectural Decision Records (ADRs) provide a lightweight, structured documentation format to record individual decisions [6]. In Nygard's established formulation, an ADR captures a title, status (e.g., Proposed, Accepted, Superseded), business and technical context, the decision made, and its resulting consequences (both positive and negative trade-offs). Despite their recognized value in maintaining architectural traceability, consistent ADR adoption in industrial practice is frequently hampered by time constraints, high cognitive overhead, and the burden of manual authoring.

Architectural Patterns for AI-Enabled Systems. Architectural patterns offer reusable, proven solutions to recurring design problems within specific operational contexts. While traditional software patterns address general modularity, loose coupling, and component interaction, AI-enabled systems introduce distinct engineering concerns. These include managing non-deterministic model inference, training and serving pipeline orchestration, data drift, continuous monitoring, and specialized compute infrastructure. In response, recent software engineering research has codified specialized catalogs and taxonomies of design patterns tailored for machine learning and AI-based applications [3], [4]. These patterns span key lifecycle stages, including data acquisition, feature preparation, model training, model deployment, serving, and runtime observability [5]. However, matching concrete system requirements to appropriate AI patterns remains an unstructured, manual task that requires deep familiarity with both the operational domain and extensive pattern repositories.

LLMs in Architecture: Retrieval versus Generative Reasoning. The emergence of Large Language Models (LLMs) with advanced natural language understanding and reasoning capabilities presents new opportunities for architectural decision support. In this context, two primary paradigms exist:

1) *Dense Semantic Retrieval (RAG):* Retrieval-Augmented Generation techniques embed requirement statements and pattern definitions into continuous vector spaces to retrieve top-ranking candidates based on semantic proximity [11]. Dense retrieval is computationally efficient and scalable across large catalogs, but operates primarily on lexical and semantic surface similarity without evaluating domain-specific trade-offs, structural constraints, or verifiable requirement satisfaction.
2) *Generative Applicability and Decision Synthesis:* Prompting frontier LLMs with explicit requirement contexts, acceptance criteria, and structured pattern specifications enables direct reasoning over compatibility. Beyond simple similarity, generative models can categorize applicability under formalized decision rules, cite explicit textual evidence from requirements, and synthesize complete, structured ADR drafts documenting context and trade-offs.

Understanding the operational relationship, overlap, and practical boundaries between dense semantic retrieval and generative reasoning is central to designing effective, requirement-grounded architectural assistance pipelines.

B. LLM-Assisted Software Architecture Design

Recent research has increasingly investigated the capabilities and boundaries of LLMs as intelligent assistants in software architecture design. Soliman and Keim [9] conducted an exploratory empirical evaluation probing whether large language models embody reliable software architectural knowledge (AK), assessing GPT responses against established ground truth on the large-scale HDFS system. While the model demonstrated moderate recall in retrieving general architectural concepts, it exhibited substantially lower precision, particularly when tasked with identifying quality attribute solutions and capturing nuanced design rationales. Their findings established that while LLMs can serve as an exploratory source for initial architectural ideas, independent expert verification remains indispensable due to the absence of reliable semantic self-grounding.

Beyond unstructured knowledge queries, researchers have explored structured, methodology-guided interactions between human architects and generative models. Cervantes et al. [7] examined the broader framing of LLMs as collaborative architectural assistants, identifying their strengths in brainstorming

design concepts, structuring documentation, and exploring technology trade-offs. To operationalize this collaboration within a formal architectural method, Cervantes et al. [12] proposed an LLM-assisted approach embedded within the Attribute-Driven Design (ADD) framework. By supplying the model with an explicit ADD method description, an architect persona, and a structured iteration plan, the system guides the LLM to co-create architectural design artifacts. Their case-study evaluations with professional architects revealed that while generated designs partially satisfy architectural drivers, continuous human oversight, explicit decision boundaries, and iterative refinement are essential to prevent ungrounded architectural drift.

These studies demonstrate the feasibility of employing LLMs across diverse architectural tasks, including conceptual design, quality-attribute reasoning, and structured method execution. However, existing work has predominantly focused on open-ended conversational prompting, broad architectural drivers, or manual prompt-response loops across general software systems. Systematic, requirement-grounded pattern selection across codified catalogs of AI-specific architectural patterns—coupled with the automated synthesis of traceable decision records—remains underexplored. Our work addresses this gap by formalizing a requirement-by-pattern census evaluation that enforces strict textual grounding rules, compares generative reasoning against dense semantic retrieval baselines, and generates verifiable, structured ADRs for professional evaluation.

C. LLM-Based ADR and Design Rationale Generation

Documenting architecture decisions through structured records (ADRs) and capturing the underlying design rationale (DR) are central challenges in architectural knowledge management. Because manual documentation is often neglected in fast-paced software development, recent studies have investigated whether LLMs can automate or support decision authoring.

Dhar et al. [8] conducted an exploratory empirical study evaluating the feasibility of generating architectural decisions using LLMs. Focusing on existing open-source ADR corpora, they formulated the task as generating the *Decision* section of an ADR conditioned on a human-authored *Context* section. They evaluated zero-shot, few-shot, and fine-tuning approaches across models including GPT-4, GPT-3.5, and Flan-T5, using automated lexical and semantic similarity metrics (BLEU, ROUGE, BERTScore) against human-written reference decisions. Their results showed that advanced models like GPT-4 can generate plausible decisions in zero-shot settings, but highlighted key limitations: lexical overlap metrics correlate poorly with architectural soundness, and the generation pipeline presupposes that an architect has already synthesized and documented an explicit decision context.

Focusing on the argumentative reasoning behind architectural choices, Zhou et al. [13] investigated LLM generation of design rationale across 100 architecture-related problems mined from Stack Overflow and GitHub issues and discussions. Evaluating five LLMs across zero-shot, Chain-of-Thought (CoT), and LLM-based agent strategies against human-expert ground truth, they observed moderate recall (0.63–0.72) but low precision (0.27–0.28) in generating trade-off arguments. While a substantial majority of generated arguments not originally mentioned by experts were deemed helpful, between 1.6% and 3.2% were potentially misleading or incorrect, underscoring the risk of hallucinated trade-offs in automated rationale recovery.

These investigations provide important baseline evidence regarding LLM competence in decision and rationale formulation. However, important differences distinguish prior efforts from the workflow evaluated in this paper:

- *Input Grounding and Granularity:* Prior studies condition generation on pre-formulated ADR context paragraphs [8] or informal forum discussions [13]. In contrast, our pipeline operates directly on structured industrial requirements (user stories and acceptance criteria) and evaluates applicability across an established catalog of codified architectural patterns.
- *Artifact Completeness:* Rather than predicting an isolated field (such as the decision statement or rationale arguments), our generator produces complete, multi-field Nygard ADRs (Title, Status, Context, Decision, and Consequences), maintaining internal consistency between cited requirement evidence and pattern trade-offs.
- *Pipeline Traceability:* We evaluate ADR generation as an explicit downstream stage conditioned on a verified applicability assessment, establishing an unbroken audit trail from functional requirements to candidate pattern selection and final decision records.

D. Positioning Against AI-System Pattern Research

The codification and operational use of architectural design patterns for AI-enabled systems represent an active and rapidly expanding research frontier. Closest to our investigation, Oliveira et al. [10] evaluated the capacity of state-of-the-art LLMs (Gemini 1.5 Flash, Claude 3.5 Sonnet, and o1-mini) to serve as "design partners" by generating architectural pattern solutions across 65 documented AI patterns from the SWE4AI repository [5]. Using a mixed-methods evaluation combining cosine similarity, expert review of divergent solutions, and blind practitioner preference, they observed that LLMs can synthesize conceptual pattern solutions that match or surpass literature-defined descriptions in over 90% of cases.

While both studies investigate architectural patterns for AI-enabled systems using overlapping pattern taxonomies derived from the SWE4AI corpus [4], [5], their research objectives, inputs, and evaluation tasks diverge fundamentally:

1) *Pattern Generation versus Requirement-Grounded Selection:* Oliveira et al. evaluated *solution synthesis in the abstract*: given a pattern's documented problem statement and intent, can an LLM generate an equivalent or improved pattern structure? In contrast, ADR4AI evaluates *requirement-driven applicability*: given concrete, uncurated industrial user stories and acceptance

criteria from four production systems, can a model discern which existing, codified patterns actually apply under strict textual evidence rules?

2) *Architectural Artifact Production:* Whereas Oliveira et al. focused on conceptual pattern descriptions, our workflow connects pattern selection directly to the automated authoring of complete Architectural Decision Records (ADRs) documenting operational context, chosen solutions, and positive and negative trade-offs.
3) *Retrieval versus Reasoning Comparison:* While prior pattern generation research assessed LLM generative capacity in isolation, our study empirically compares generative applicability reasoning against a standard dense semantic retrieval (RAG) baseline, quantifying overlap and divergence across an exhaustive census of 840 story-pattern pairs.

From a complementary perspective, automated pattern identification has also been investigated in retrospective code analysis. Jánki and Bilicki [14] demonstrated that GPT models guided by formal rule systems can achieve 90% accuracy in detecting Model-View-Whatever (MVW) architectural patterns within Angular web application source code. Their work addresses pattern *recognition* in existing software artifacts based on deterministic structural rules. By contrast, ADR4AI addresses *prospective selection and architectural decision documentation* during requirements engineering and early design, before code artifacts exist.

Research Gap. In summary, while prior research has probed LLM architectural knowledge [9], method-guided design assistance [7], [12], isolated ADR decision completion [8], rationale argument recovery [13], abstract pattern solution synthesis [10], and code-level pattern recognition [14], no prior study provides: (i) an empirical evaluation of requirement-grounded architectural pattern selection over a codified catalog of AI-enabled design patterns; (ii) a systematic comparative analysis between generative LLM reasoning and dense semantic retrieval across an exhaustive census of industrial requirements; and (iii) an end-to-end, traceable pipeline that transforms requirement-grounded pattern selections into complete, inspectable Architectural Decision Records evaluated by practicing software architects. This study directly addresses this research gap.

III. Methodology

A. Research Questions and Study Scope

We organize the study around three research questions (Table I). The computational protocol treats a user story and its acceptance criteria as a requirement case, with the system purpose providing context. It assesses each catalog pattern against each case independently. The assessment distinguishes applicability, non-applicability, and insufficient information; a positive classification requires a supporting user-story or acceptance-criterion reference. These classifications are model outputs, not independently established ground truth.

RQ1 characterizes the model's classification outcomes rather than measuring their correctness. RQ2 compares the patterns classified as applicable with a separate retrieval baseline. This baseline retrieves catalog candidates and reranks them to a final Top-5 without making an LLM-based applicability judgment. Overlap is therefore a descriptive relationship between two output sets, not evidence of either approach's precision, recall, or superiority.

ADR generation is a separate stage: each pair classified as applicable supplies the requirement case, selected pattern, and applicability evidence for one generated ADR. Generation completeness and stored provenance establish the availability and traceability of these records, not their architectural quality. RQ3 examines expert assessments of these selected pairs and their corresponding ADRs, rather than adding another automatic applicability classification.

The prepared questionnaire measures decision adequacy for the stated requirements (Q1), decision grounding (Q2), presentation of relevant consequences and trade-offs (Q3), perceived usefulness as a decision record (Q4), and pattern applicability (Q5). These are five separate expert-assessed dimensions; they are not a validated composite quality score. Optional comments provide context for the ratings. Neither implementation feasibility, cost savings, novelty, nor actual improvement in architectural practice is directly measured by these items.

Experts evaluate cases for a system they know, and only pairs selected by the LLM are presented. Consequently, RQ3 can characterize expert judgments of the presented recommendations but cannot identify missed applicable patterns or estimate recall over the full set of assessed pairs. Expert agreement with a recommendation does not by itself establish implementation success or objective correctness. Repeated ratings from an expert and multiple cases from a system must not be treated as independent replications when interpreting the results. Conclusions remain bounded to the sampled requirements, catalog, generated records, and participating experts.

B. Industrial Requirement Sample

The study uses a bounded industrial sample comprising 4 systems, 12 user stories (3 per system), and 35 acceptance criteria (Table II). The supplied data-scope document defines this as a minimal, delimited sample for studying architectural-pattern selection and decision documentation for AI-enabled systems. It is not a representative sample of industrial requirements or AI-enabled systems. The available artifacts do not establish a probabilistic sampling procedure; we therefore do not interpret these counts as population estimates.

Each requirement case consists of the system purpose, one stakeholder user story, and all of that story's acceptance criteria. The source CSV stores one acceptance criterion per row, repeating the corresponding purpose and story. Grouping by the stable user-story identifier reconstructs the case without rewriting the original Portuguese requirement text or changing criterion order. The criterion-row count must not be confused with the number of requirement cases.

TABLE I
RESEARCH QUESTIONS, UNITS OF ANALYSIS, AND EVIDENCE MAPPING.

ID	Research question	Unit, evidence, and intended analysis
RQ1	How does the LLM classify architectural-pattern applicability across the requirement cases, and which patterns does it identify as applicable?	Unit: one user-story–pattern pair. Evidence: the official exhaustive assessment log, original requirements, and pattern catalog. Analysis: distributions of the three classifications and applicable-pattern sets by story and system, retaining zero-positive stories.
RQ2	To what extent do the patterns classified as applicable by the LLM overlap with the frozen RAG baseline's final Top-5 candidates for the same requirement cases?	Unit: one LLM-positive user-story–pattern pair, grouped by story. Evidence: official applicability assessments and the frozen final Top-5 rankings. Analysis: membership-intersection counts by story and overall, with the LLM-positive set as an explicit denominator; no applicability-accuracy metric.
RQ3	How do experts assess the applicability of the LLM-selected patterns and the adequacy, grounding, consequences, and perceived usefulness of the corresponding generated ADRs?	Unit: one expert's rating of one selected pair and its ADR on one questionnaire item. Evidence: consolidated expert responses to Q1–Q5 and optional comments for the prepared cases. Analysis: item-specific ordinal distributions and case-level summaries with response counts; comments contextualize ratings. Aggregation and inclusion rules must be specified before consolidated analysis.

TABLE II
THE BOUNDED INDUSTRIAL REQUIREMENT SAMPLE.

System ID	User stories	Acceptance criteria
SYS01	3	6
SYS02	3	11
SYS03	3	7
SYS04	3	11
Total	12	35

System purpose provides contextual information, but is not sufficient on its own to establish pattern applicability. Positive evidence must be grounded in the user story and/or its acceptance criteria. The model is not authorized to infer unstated technologies, requirements, or constraints from the purpose.

The source uses generic names `System_01`–`System_04`; the paper uses the corresponding stable identifiers `SYS01`–`SYS04`. Story and criterion identifiers preserve traceability within each system without exposing real system names. The organization is not named in this review draft. According to the supplied scope document, the study input is restricted to purposes, identifiers, user stories, and acceptance criteria; it does not include operational databases, personal data, credentials, customer records, or production content. This delimitation does not itself establish participant consent, ethics approval, or permission for unrestricted public release.

C. Architectural Pattern Catalog

We use 70 pattern records from the repository associated with Heiland et al.'s multivocal literature review of design patterns for AI-based systems [4], [5]. For provenance verification, we compared the local files with the public repository at revision `9ff3e61` (the full revision is recorded in the artifact audit). All 70 records matched the corresponding repository files byte-for-byte, as checked using Git blob hashes. This identifies the source snapshot of the catalog; it does not establish the effectiveness of the individual patterns.

Each pattern is stored as one JSON record with a stable three-digit identifier, from `001` through `070`. The applicability prompt supplies exactly five catalog fields: *ID*, identifying the record; *name*, naming the pattern; *motivation*, describing the problem or need; *solution*, describing its proposed response; and *consequences*, describing resulting effects and trade-offs. Identifiers remain strings, preserving leading zeros and enabling joins with the assessment and ADR artifacts. Source text is preserved rather than normalized or rewritten.

The records also contain aliases, examples, related-pattern identifiers, categories, and resource identifiers. These additional fields are not supplied to the applicability model. Resource identifiers provide source pointers within the repository, not independently verified bibliographic descriptions in the local catalog. We therefore attribute the collection to the verified repository and its associated publication rather than infer individual source citations from numeric resource codes.

All five input fields are present as strings in every record. Explicitly empty motivation or consequences strings are retained as empty inputs, without inventing replacement descriptions. A missing field or non-string value is an input-validation failure, not an invitation to repair the source. This preservation policy keeps the assessed catalog traceable while acknowledging uneven descriptive completeness.

The catalog includes general architectural patterns, such as Client–Server, alongside patterns concerning machine-learning safeguards and model serving. We use the records as a bounded candidate set, not as equally mature, mutually independent, or universally applicable solutions. Catalog inclusion alone does not establish applicability to a requirement case; that relationship is assessed separately.

D. Exhaustive Pattern-Applicability Assessment

The experiment crosses all 12 requirement cases with all 70 catalog patterns, yielding $12 \times 70 = 840$ planned pairwise assessments. Each request contains one complete requirement case and one pattern, with no prior assessment or conversation history supplied. This exhaustive design does not retrieve or

prefilter candidates. Separate requests do not imply statistical independence: assessments share stories, systems, catalog entries, and a model. The resulting classifications are model predictions, not expert-provided ground-truth labels.

a) Model and structured response: The recorded model identifier is `gpt-6-sol`, with reasoning effort `high`. The implementation uses the Responses API with strict JSON-schema output named `pattern_assessment`. The schema requires exactly three fields: `classification` (one of the three labels below), `requirement_refs` (an array of strings), and `rationale` (a string); additional properties are disallowed. Each stored result also includes the story ID, pattern ID, model identifier, and reasoning effort. No temperature or seed is set in the request, and the recorded model identifier does not pin a dated model snapshot.

b) Decision and grounding rules: The instructions distinguish:

- `applicable`: the story and/or its acceptance criteria provide positive evidence of a need, problem, constraint, or quality concern addressed by the pattern.
- `not_applicable`: those requirements provide evidence that the pattern does not fit the stated need, problem, constraint, or context.
- `insufficient_information`: the story and criteria do not provide enough evidence to establish either applicability or non-applicability.

System purpose supplies context but cannot alone justify applicability. The prompt prohibits assuming unstated requirements, technologies, constraints, or system characteristics. Absence of contradiction, possible usefulness, or the possibility of implementing, testing, or improving a requirement using a pattern is not sufficient positive evidence. Assessment concerns underlying architectural intent rather than literal terminology. An applicable response must cite at least one identifier belonging to the current story or one of its criteria, and briefly explain the evidence–pattern relationship; it must not propose an architecture, implementation, technology, or new requirement. These are prompting and reference-validation constraints, not proof that a model's rationale is semantically correct.

c) Execution, retries, and validation: Execution is sequential, in CSV story order and ascending pattern-ID order. SDK retries are disabled. The runner permits at most 3 attempts per pair, revisiting unresolved pairs in up to 3 ordered passes rather than immediately retrying each failed request. Before a request, it atomically persists an attempt reservation, so interrupted requests consume the attempt budget. A request failure updates that entry with its error type and a whitespace-normalized message, truncated to 300 characters with the configured API key redacted. Successful validation removes the pair from this unresolved-failure state.

The response must have completed, contain no refusal, and provide parseable structured output. Local validation checks the exact fields, label, reference array and identifier membership, a nonempty reference array for `applicable`, and a string-valued rationale. These checks do not require a nonempty rationale string or test the factual entailment of cited text. Only validated results are appended to the official log, flushed, and synced to disk; storage failures stop execution rather than risk duplicate writes. A nonblocking file lock prevents concurrent official runners.

On resume, the runner validates existing JSONL records, model configuration, pair uniqueness, and failure-state attempt counts. Malformed records, unknown identifiers, and interrupted writes lacking a final newline stop the run rather than being repaired or discarded. Completed pairs are skipped; unresolved attempt counts are retained, and exhausted pairs are not retried. The final on-disk check requires exactly all 840 expected pairs and 70 records per story before exporting story-specific applicable-pattern selections.

d) Official dataset and replication artifact: The official dataset is the top-level `assessments.jsonl` under the pattern-selection experiment's `outputs` directory, accompanied by `failures.jsonl`. Separate `pilot_SYS01-US01` directories (including the `_v1` and `_v2` variants) and `test_assessment.json` and `test_assessment_v2.json` are excluded, not merged with official records. An offline audit verifies complete official coverage and records input/source hashes, the exact instructions, JSON schema, and input template. The template presents system purpose, identified story and criteria, followed by the five pattern fields in Section III-C; the full instructions remain in the artifact rather than this space-bounded description.

The frozen log contains 840 validated pair outcomes and an empty final unresolved-failure file. This establishes completion, not that no retries occurred: successful records omit attempt counts, and successful retries remove failure-state entries. Thus, 840 describes the planned first-pass requests and retained outcomes, not an independently verified total number of API attempts. We do not claim exact stochastic reproduction from the stored configuration.

E. Retrieval and Reranking Baseline

We compare the LLM-positive pairs with a separate retrieval/reranking baseline, called the RAG baseline in the artifacts. Despite that name, this pipeline performs no generative step: it uses no external LLM to classify applicability or generate ADRs. Its final rankings are candidate recommendations, not validated applicability judgments.

a) Query and catalog representation: The baseline groups the English requirement CSV by story ID, retaining the CSV encounter order. One query per story concatenates labeled system purpose, the user story preceded by its bracketed identifier, and all acceptance criteria, each preceded by its bracketed identifier. Identifiers are preserved in the query and output for traceability, not substituted for requirement text. An offline audit verifies that the story/criterion identifiers and their ownership match the original sample. The baseline uses the English requirement copy, whereas the applicability experiment uses the original Portuguese text; this is not a language-controlled comparison, and identifier alignment does not itself establish translation equivalence.

The retrieval corpus contains one document per catalog pattern. It includes the pattern ID and nonblank source values for name, aliases, motivation, solution, consequences, examples, categories, related patterns, and resources; list entries are joined with semicolons, omitting blank entries. These catalog records match the original pattern fields, but the retrieval representation is richer than the five-field applicability input. The Chroma collection is versioned by a fingerprint of catalog content, text-format version, and embedding-model identifier. Index compatibility checks verify complete IDs, document text, and metadata before reuse, rebuilding incompatible contents.

b) Vector candidates and final ranking: The embedding model is `BAAI/bge-small-en-v1.5`. Chroma vector similarity search retrieves 15 candidate documents per query (vector Top-15), using the library's default similarity configuration without overriding the distance metric. The cross-encoder `BAAI/bge-reranker-base` scores each query–candidate pair. Candidates are sorted by descending reranker score, with vector rank breaking ties, to retain the final Top-5. The implementation requires exactly 15 vector candidates and one finite reranker score per candidate. Thus, Top-5 is a reranked subset of Top-15, not the first five vector results. The stored output preserves the vector ordering and the final reranked ordering separately.

c) Candidate-overlap measure: Let S be the set of sampled stories, A_s the set of pattern IDs classified as `applicable` by the LLM for story s, and R_s the set of IDs in the frozen final Top-5 for that same story. For each LLM-positive pair (s, p), define $m(s, p) = 1$ when $p \in R_s$, and $m(s, p) = 0$ otherwise. The per-story overlap count is $o_s = |A_s \cap R_s|$. The pooled LLM-positive-set overlap proportion is

$$O = \frac{\sum_{s \in S} |A_s \cap R_s|}{\sum_{s \in S} |A_s|}, \qquad \sum_{s \in S} |A_s| > 0. \tag{1}$$

This weights positive pairs, not stories equally; it is not an average of story-level percentages. A zero-positive story remains in the sample with $o_s = 0$, but contributes neither a positive pair nor a per-story percentage. If no story has a positive pair, O is undefined, not zero. Only final Top-5 membership enters this measure: vector Top-15 membership, ranking position, and reranker scores do not. Since A_s consists of model predictions rather than independently established truth, O measures candidate overlap, not precision, recall, accuracy, ground-truth applicability, or superiority of either approach. A nonmatch does not demonstrate a missed applicable pattern.

d) Frozen source and reproducibility limits: The comparison uses `rag_baseline.csv` in the RAG experiment's `outputs` directory, not the separate `rag_baseline_reproduced.csv`. An offline audit validates one frozen row per story, unique known IDs, ID/name alignment, and final Top-5 inclusion in vector Top-15, without recomputing rankings. The package records fixed model identifiers and pins direct dependencies and numerical runtimes, but does not pin downloaded model revisions or fully lock transitive dependencies. Rebuilt approximate vector indices and numerical hardware differences may change ranks. Catalog/text compatibility checks do not eliminate these sources of variation. No embedding models, reranker, or index were executed for this manuscript audit, and exact end-to-end reproduction is not established by these structural checks.

F. Architecture Decision Record Generation

Figure 1 summarizes the pipeline and its separate retrieval comparison branch. ADR generation consumes the 36 unique (`user_story_id`, `pattern_id`) pairs classified as `applicable`. It produces one proposed record per pair, not a jointly optimized architecture or an expert-approved decision. The accepted protocol pilot is included in the 36 records.

a) Inputs and prompting: Each request supplies the system purpose, identified story and all criteria; the selected pattern's ID, name, motivation, solution, and consequences; and the prior applicability assessment's evidence references, corresponding requirement text, and rationale. RAG rankings and overlap flags are excluded. The prompt permits legitimate empty catalog descriptions and limits generation to one contextualized architectural decision. It prohibits inventing unsupported requirements, technologies, components, protocols, or implementation details, or merely repeating that the system will adopt the named pattern. System purpose is context, while the story and criteria supply requirements. The proposed decision must instantiate the pattern's architectural intent; consequences must follow from the supplied information rather than portray the decision as universally beneficial.

The Nygard-inspired structure [6] comprises a short title, neutral context describing the relevant forces, an active-voice decision phrased as "We will …", and relevant positive, negative, and neutral consequences. These are prompting rules, not automatically verified semantic properties. One synthetic recruitment/compliance example demonstrates the transformation using catalog Pattern `007`, which is outside the selected ADR-generation pairs. The example is instructional, not an empirical case. The generated ADR must not mention the model, experiment, applicability process, or evidence identifiers. Source references and rationale remain in the surrounding research artifacts for traceability.

b) Configuration and output validation: Generation uses `gpt-6-sol` with reasoning effort `high` and one independent Responses API request per ADR, without prior generated ADRs in its input. The client disables SDK retries and sets a 600-second timeout. No temperature or seed is supplied, and the model identifier is not a dated snapshot. Strict JSON-schema output, named `adr_pilot` for both pilot and full execution, requires exactly four string-valued fields: title, context, decision, and consequences, disallowing additional properties. Local validation rejects incomplete responses, refusals, duplicate JSON keys, incorrect field sets, and empty or non-string fields. The application then adds the fixed status `Proposed`; the model neither selects nor outputs that status.

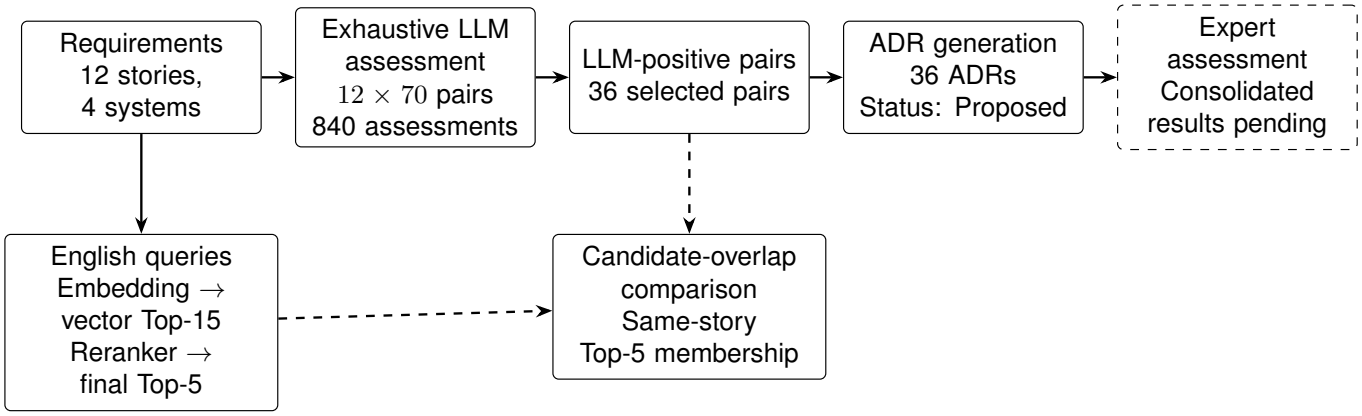


Fig. 1. ADR4AI pipeline. The retrieval branch supplies candidate rankings for comparison, not ADR-generation input. Dashed comparison arrows denote final-Top-5 membership; the dashed expert stage has no consolidated results in this draft.

Neither schema compliance nor a Proposed status establishes architectural adequacy or stakeholder acceptance.

Each completed JSON artifact stores its pair identifiers, model and reasoning effort, response ID, `api_calls=1`, source-file path and SHA-256, rendered-prompt SHA-256, strict-output validation flags, and the five ADR fields. The source hash identifies the pre-ADR selected-pair CSV bytes, now retained in the backup: the current selected-pair CSV additionally contains an ADR column and therefore has a different file hash. The offline compiler validates pair coverage, metadata, source/prompt hashes, and fields before joining outputs by pair IDs. ADR text is retained as generated, not normalized, translated, or architecturally corrected.

c) Pilot, execution, and authorized retry: The research artifacts document approval of the `SYS01-US01`–Pattern `001` pilot. Full execution reuses that record without regenerating it and processes only pending pairs sequentially. It validates existing outputs and refuses duplicate or overwritten completed records. Persistent attempt claims prevent silent retries after interruptions; failure records retain the error, and any failure stops execution. Publishing full-run artifacts uses exclusive atomic creation rather than replacement.

For `SYS04-US03`–Pattern `003`, the archived failure reports "Connection error." A second attempt explicitly records manual retry authorization and links to the original claim. Both claims and the failure remain execution history, separate from the one completed ADR for that pair. Because a connection failure may precede dispatch, this failed attempt cannot be equated with an additional completed model response. The pilot plus final outputs cover exactly 36 unique selected pairs; attempt metadata is not counted as additional ADRs. Pilot approval concerned the generation protocol, not the future experts' judgments. Consolidated expert assessment is still pending in this draft.

G. Translation and Expert-Evaluation Preparation

a) Completed preparation: Portuguese presentation material: Evaluator-facing material was prepared for the 36 selected pairs. The original system purpose, story, and acceptance criteria are retained literally; translation operates on four pattern fields (name, motivation, solution, consequences) and five ADR fields (title, status, context, decision, consequences), yielding 324 field mappings. The linguistic inventory distinguishes 12 Portuguese fields preserved, 289 non-Portuguese fields translated, 14 mixed-language fields translated partially, and nine empty fields preserved. The translation prompt prohibits improving, completing, or architecturally correcting the source. Portuguese portions of mixed fields are protected by placeholders and restored locally; the fixed status `Proposed` is presented as *Proposto*. The preparation report records 36 translation requests in the prior translation stage; the subsequent review/compilation stage made no additional model calls.

Five field-specific linguistic corrections were explicitly authorized: "Commercial" to "Comercial" in four ADR fields for `SYS03-US02`–Pattern `014`, and "Machine Learning" to "aprendizado de máquina" in the pattern motivation for `SYS04-US03`–Pattern `014`. Original text, translation candidate, final presentation, action category, and correction authorization remain separate in the audit. An offline compiler checks exact source/mapping alignment, preserved Portuguese spans, authorized changes, requirement preservation, and pair equality before producing the presentation CSV. These checks establish linguistic provenance and preservation constraints, not architectural validation or independently measured translation quality.

b) Prepared instrument: items and routing: The archived Google Apps Script embeds all presentation cases verbatim. Three required profile questions ask about professional experience in software development/architecture/engineering, architecture-specific experience, and self-rated knowledge of the evaluated system. Both experience items use four categories: less than two years, two–five, six–ten, and more than ten; system knowledge uses a 1–5 scale from very low to very high. These items describe respondents but do not themselves enforce expert eligibility. The instructions ask evaluators to

TABLE III
PREPARED QUESTIONNAIRE CONSTRUCTS, IN PRESENTATION ORDER. ENGLISH DESCRIPTIONS OF THE PORTUGUESE ITEMS; NOT OBSERVED OUTCOMES.

Item	Judgment requested
Q1	Adequacy of the proposed decision for the story and acceptance criteria.
Q2	Sufficient grounding of the architectural decision to understand why it was proposed in that context.
Q3	Adequate presentation of relevant consequences, including benefits and possible impacts or trade-offs.
Q4	Usefulness, in the evaluator's system, as a record supporting understanding, discussion, or evolution of the decision.
Q5	Applicability of the underlying pattern to the story and acceptance criteria.

select only the system they know or have experience with; the required single-choice selection routes each evaluator to that system and then to the common final page, not onward to another system.

The prepared routes contain 7, 12, 8, and 9 cases for SYS01–SYS04, respectively. Within a route, purpose, story, and criteria are displayed once per story, followed by each pattern and ADR. Empty pattern descriptions are omitted from display without filling their stored values. Q1–Q4 address the ADR and Q5 its underlying pattern (Table III). All five ratings per case are required on the selected route, using a 1–5 agreement scale anchored by *Discordo totalmente* (strongly disagree) and *Concordo totalmente* (strongly agree); no verbal anchors are assigned to the intermediate values. Each case also has an optional paragraph comment asking about problems or needed improvements. Question titles encode the story ID, pattern ID, and item ID for traceable export. This yields 35, 60, 40, or 45 required case ratings per evaluator's route, not 180 ratings per evaluator. Unselected-route response columns are structurally blank, not evidence that the evaluator omitted selected-route ratings.

The five items are separate professional judgments, not a validated composite score. In particular, Q4 measures perceived usefulness as a record, not observed improvements in practice; Q5 cannot establish missed-pattern recall because only LLM-positive pairs are presented.

c) Implementation checks and unconfirmed administration: The questionnaire README reports local stand-in tests of item construction, embedded-case equality, and routing. Those simulations validate implementation, not expert judgments, Google deployment, live appearance, or account permissions. The manuscript audit independently checks frozen translation outputs, embedded values, and static questionnaire source contracts; it does not rerun the Google service or establish live routing behavior. Creation is disabled in the archived script until deliberately enabled by a researcher. The script requests no name, e-mail, or employee identifier, disables e-mail collection, response summaries, and question shuffling, and does not restrict submissions to one per signed-in user. These are intended settings, not a guarantee of anonymity: organization policies and responder permissions require account-side review and signed-out access testing. Profile combinations and free-text comments may also permit indirect identification.

d) Pending protocol and post-collection confirmation: **Protocol details pending confirmation:** document the deployed form/version and live route/access checks; evaluator eligibility and recruitment; consent and the applicable ethics approval or exemption; administration dates and instructions; and actual privacy, response-storage access, and retention arrangements. The preparatory research-purpose notice does not itself establish informed consent or ethics approval. The archived scripts alone do not establish that a form was deployed or that responses were collected.

The researcher reports preliminary responses being consolidated separately. No consolidated response dataset or verified administration record has been inspected for this draft. After collection, replace the pending procedural details with confirmed execution information, documenting any differences from the prepared instrument. This subsection contains no expert-result values or claims of agreement. Response eligibility, ordinal analysis, and comment handling are specified in Section III-H before response inspection.

H. Pre-Inspection Expert-Analysis Protocol

We record this exploratory, descriptive protocol before inspecting expert responses for manuscript analysis. Since the researcher reports preliminary responses already collected elsewhere, this is not a preregistration before collection. The versioned protocol records its timing, choices, output inventory, and future amendments; any change must disclose its reason and whether it followed exposure to results. Table IV predeclares the expert-result outputs. No response values are analyzed here.

a) Inclusion, routing, and missingness: Include submissions only after documented consent or applicable participation authorization, familiarity with the chosen system, and one recognized route with a compatible administered instrument are confirmed. Restricted administration records may supply confirmations absent from the form. No minimum years of experience or self-rated knowledge cutoff is imposed; profile questions alone do not prove eligibility. Unresolved eligibility is held pending, not inferred from favorable ratings. Retain an immutable raw-to-derived disposition log with included, excluded, and held-pending counts reconciling to the raw export.

Remove only proven duplicate export rows or submissions using provenance, not matching rating vectors or profiles. Hold unresolved duplicates and route conflicts pending investigation; do not silently select the more favorable response. Anonymous submissions are not necessarily unique people: report submission counts separately from confirmed distinct evaluator counts. Missing profile values do not remove otherwise eligible ratings.

Primary analysis retains available valid selected-route items from eligible submissions. Selected-route blanks and invalid values are recorded separately; neither is imputed. A valid

TABLE IV
PREDECLARED EXPERT-ANALYSIS OUTPUT INVENTORY. THESE ARE PLANNED OUTPUTS, NOT OBSERVATIONS; AGGREGATE PUBLICATION REQUIRES DISCLOSURE REVIEW.

Output	Source and computation
Sample	Disposition log, profiles and administration confirmations. Reconcile row statuses; count route submissions, confirmed evaluators and profile categories with valid/unknown denominators.
Items	Eligible ratings and case-story-system mapping. System and balanced study distributions per item using the specified hierarchy; inverse-CDF median/quartiles, raw valid n and coverage.
Cases	Eligible ratings, cell flags and prepared case/item grid. Every case/item: opportunities, valid/missing/invalid n, category counts/percentages and quantiles; no-data summaries unavailable.
Hierarchy	Case/item distributions and prepared mapping. Equal-case story and equal-story system distributions, quantiles and observed/expected coverage; never average medians.
Figure	Primary item table's distributions. Stacked bars of all five ordered categories by system and balanced study summary, with coverage and identical scales.
Agreement	Confirmed-rater IDs joined to eligible ratings. Within-system/item ordinal alpha using the specified coincidence formula and gate; report coverage or non-estimability.
Sensitivity	Eligible complete-route submissions. Repeat case, hierarchy and item summaries unchanged; compare distributions and coverage with primary analysis.
Themes	Eligible comments, codebook, independent coding and reconciliation logs. Unique comments per theme with comment/opportunity denominators; preserve negative cases and multiple-code accounting.

rating is one integer category from 1 through 5, after whitespace trimming, without rounding or clipping. Blank cells on unselected routes are structural, outside eligible opportunities. Nonblank off-route ratings or ambiguous route selection quarantine the submission until export/instrument mapping is resolved. For each case/item, valid, missing, and invalid counts must sum to eligible opportunities. Repeat the analysis on complete-route submissions as a sensitivity check, reporting changed coverage even if no such submission remains.

b) Ordinal summaries and balanced aggregation: Analyze Q1–Q5 separately, without means, a composite score, significance tests, or treating ratings as independent participants. Report counts and percentages for all five categories among valid items, alongside opportunity, missing, invalid, and valid-response denominators. Use the non-interpolated inverse empirical distribution for the median and quartiles: the smallest category whose cumulative proportion reaches 0.50, 0.25, or 0.75, respectively. For even sample sizes this yields the lower category median rather than an arithmetic midpoint. Display the median with its interquartile category interval $[Q_{0.25}, Q_{0.75}]$, without assuming equally spaced ratings. With no valid rating, summaries are unavailable, not zero.

First compute case/item distributions, giving each retained valid submission one vote within a case. Average observed case distributions equally within a story, story distributions equally within a system, and system distributions equally for the study summary. Apply the same quantile rule to these distributions, not to averaged medians. This avoids automatically overweighting systems with more recommended patterns. Report contributing and absent cases, stories, and systems; an observed-system summary with partial coverage must not imply all 4 systems were assessed. Raw counts and balanced percentages have different weighting and must be labeled. Retain item-wise within-submission summaries internally, without publishing respondent-level links to profiles. Stories with no selected pairs create no expert-rating opportunities and are not zero-score cases.

c) Conditional inter-rater agreement: Plan ordinal Krippendorff's alpha separately for each system and item, with ADR cases as units and confirmed distinct evaluators as raters. Do not pool raters assigned to different systems or equate unverified anonymous submissions with distinct raters. Compute alpha only when at least two confirmed raters share a case and expected disagreement is positive. The artifact specifies ordered-pair coincidence weighting, ordinal distance from empirical category marginals, and $\alpha = 1 - D_o/D_e$. Report rater and pairable-case counts, rating coverage, and any non-estimability reason; a single rater or invariant ratings do not justify an invented coefficient. Agreement is exploratory consistency, not correctness or agreement with the study hypothesis; no acceptance threshold is applied.

d) Comments and interpretation: Use each eligible nonblank case comment as the qualitative unit. Two researchers independently code the comments and reconcile differences, retaining initial assignments, codebook versions, and reconciliation reasons. Applicability, adequacy, grounding, consequences, usefulness, clarity, feasibility, missing context, and novelty are initial sensitizing domains, not guaranteed findings or additional questionnaire scales. Allow inductive subcodes and multiple codes, recoding affected comments after codebook changes. Preserve critical, mixed, and contradictory observations. Report unique comment counts per theme and comment opportunities; multiple themes may exceed the nonblank-comment total. Blank optional comments are not negative ratings. Quotations require documented authorization and de-identification; otherwise summarize without quoting. If dual coding cannot be executed, disclose the actual procedure and deviation.

There is no binary "expert-validated" criterion, favorable-rating cutoff, majority success rule, or post hoc threshold. Interpret the actual item-specific distributions and reservations with their coverage, distinguishing ADR judgments

from Q5 applicability. They cannot establish missed-pattern recall, implementation success, or causal improvements in practice. All publication outputs require disclosure review; privacy decisions must not selectively suppress criticism. Consent/governance, instrument-version confirmation, and respondent provenance remain prerequisites to actual analysis. Phase 3 will implement and test the calculations against the frozen response export; this task records choices only.

I. Ethics, Data Governance, and Reproducibility

a) Authorization and confidentiality: The supplied industrial data-scope document defines the intended academic use of the bounded requirement sample and requires generic system identifiers. It is not evidence of formal organizational authorization, permission to transmit requirements to an external model provider, or permission for public redistribution. **Confirmation pending:** document the responsible organization's authorization, permitted processing and publication scope, and any conditions on reuse. Generic identifiers conceal real system names but do not establish irreversible anonymization: requirement text and context may still identify the organization. Until disclosure is authorized, treat source requirements and requirement-bearing prompts, model outputs, translations, and evaluator cases as restricted; derived content does not automatically lose its source restrictions.

b) Expert participation and privacy: No formal ethics-board approval, exemption, or completed consent procedure is established by the supplied artifacts. **Confirmation pending:** record the applicable ethics determination, participant information and documented consent or participation authorization, recruitment and voluntariness safeguards, withdrawal conditions, actual platform access, authorized custodians, and retention/deletion arrangements. These conditions must be resolved before response analysis; a retrospective record must not be described as prior approval. The prepared form's settings do not prove anonymous administration. Raw exports, timestamps, profiles, comments, eligibility records, and any identity-linking keys require controlled access. Keep linking keys separate from analysis copies; generic submission IDs are pseudonyms, not proof of anonymity. Review small cells, combined profiles, and free text for deductive disclosure before publishing aggregates or authorized quotations, without selectively removing criticism.

c) External models and generated content: The archived applicability and ADR implementations use the OpenAI Responses API with the recorded `gpt-6-sol` identifier and high reasoning effort; the translation audit records model-assisted presentation preparation. The retrieval branch uses embedding and reranking models, not an LLM generation call. Requirement-bearing generation inputs and outputs must therefore be covered by organizational processing permission. **Confirmation pending:** verify the provider/account conditions in force at execution, including retention, training use, access, processing location, and any contractual safeguards; no zero-retention or non-training claim is inferred from source code. This manuscript-preparation step makes no model-generation calls. Generated classifications remain model judgments and ADRs remain `Proposed`; schema checks and translation corrections do not establish semantic correctness or deployment approval. Human review remains necessary before operational use.

d) Versioning and reproducibility: Offline audits preserve source hashes, stable join identifiers, recorded configuration, prompt provenance, and output checks; deterministic compilers reconstruct derived datasets from frozen computational records without reissuing model requests. These checks establish lineage and structural consistency, not semantic validity. For expert data, the planned procedure is to preserve a read-only original export in approved restricted storage, record its checksum and instrument version, and create separately versioned cleaning, disposition, coding, and analysis outputs. Never overwrite the raw export; retain correction reasons and protocol amendments. Do not place respondent-level data or linking keys in the manuscript or public repository. Storage controls, custodians, and retention are not yet verified. Compiler determinism does not imply deterministic model regeneration: external model revisions, unpinned dependencies, approximate retrieval, and hardware changes constrain exact end-to-end replication.

e) Sharing boundaries: The planned artifact package may contain disclosure-reviewed manuscript sources, processing code, analysis specifications, sanitized prompt templates, and permitted aggregate outputs. Public catalog provenance does not alone establish redistribution rights; verify source licenses and attribution. Requirements and dependent artifacts require explicit release permission and disclosure review. Raw expert data and identity-linking records remain restricted; approved de-identified derivatives are separate release candidates, not substitutes silently presented as raw evidence. The initial collection, including its identifying scope document, is not an anonymous review package. Review files, metadata, local paths, API response identifiers, and repository history before release. No public artifact URL or completed access-control implementation is claimed here; the availability statement will identify actual releases and restrictions once verified.

IV. Results

A. RQ1: Applicability-Classification Outcomes

The exhaustive assessment produced 840 unique story–pattern outcomes. Of these, 36 (4.29%) were classified as applicable, 31 (3.69%) as not applicable, and 773 (92.02%) as insufficient information (Table V). Each percentage uses all assessed pairs as its denominator. These are model-label frequencies, not estimates of classification accuracy or expert-confirmed applicability.

Applicable outcomes occurred in 11 of 12 stories. Counts ranged from 0 to 7 per story, with each story assessed against 70 patterns. `SYS04-US01` had no applicable outcome; this is a retained classification result, not an execution failure or evidence that no pattern could apply in practice. The largest positive set belonged to `SYS04-US03`, with 7 applicable outcomes.

TABLE V
RQ1 MODEL-CLASSIFICATION OUTCOMES. PERCENTAGES USE ASSESSED PAIRS IN THE INDICATED ROW, NOT EXPERT JUDGMENTS.

Overall classification distribution			
Classification	Count	Pairs	%
Applicable	36	840	4.29
Not applicable	31	840	3.69
Insufficient information	773	840	92.02
Applicable outcomes by story and system			
Story / subtotal	Applicable	Pairs	%
SYS01-US01	3	70	4.29
SYS01-US02	1	70	1.43
SYS01-US03	3	70	4.29
SYS01 subtotal	7	210	3.33
SYS02-US01	5	70	7.14
SYS02-US02	5	70	7.14
SYS02-US03	2	70	2.86
SYS02 subtotal	12	210	5.71
SYS03-US01	1	70	1.43
SYS03-US02	5	70	7.14
SYS03-US03	2	70	2.86
SYS03 subtotal	8	210	3.81
SYS04-US01	0	70	0.00
SYS04-US02	2	70	2.86
SYS04-US03	7	70	10.00
SYS04 subtotal	9	210	4.29
All systems	36	840	4.29

At system level, applicable counts were 7, 12, 8, and 9 for SYS01–SYS04, respectively. System percentages in the table use the assessed pairs within that system, rather than the total positive set. The insufficient-information category accounted for most outcomes overall. The observed differences describe this bounded sample; they do not identify causes, establish that the requirements are defective, or imply that non-positive outcomes are missed applicable patterns. Architectural adequacy requires separate evidence.

B. RQ2: Overlap with the Retrieval Baseline

Of the 36 LLM-positive story–pattern pairs, 5 appeared in the frozen baseline's final Top-5 for the same story, yielding a pooled overlap of 13.89% (Equation 1). These matches spanned 5 of 12 sampled stories. The remaining 31 positive pairs were outside that story's final Top-5; they are not established false negatives.

Table VI retains every sampled story and reports the positive-set size, pairs inside and outside the final Top-5, and the overlap percentage among that story's LLM-positive pairs. For `SYS04-US01`, the positive set is empty: the intersection count is zero, but its percentage is undefined, not zero. The pooled percentage divides total matches by all positive pairs; it is not the average of the story percentages or a proportion of retrieval candidates.

This comparison concerns final Top-5 membership only; vector Top-15 membership, ranking positions, and scores do not contribute to this measure. The LLM labels are predictions rather than an independent reference set, so the intersection does not measure precision, recall, classification accuracy, or superiority of either approach. As documented in Section III-E, the baseline uses English queries and a richer catalog representation than the Portuguese-input applicability assessment. Consequently, observed overlap differences cannot be attributed solely to the selection mechanism.

TABLE VI
RQ2 FINAL TOP-5 MEMBERSHIP AMONG LLM-POSITIVE PAIRS. PERCENTAGE DENOMINATOR: POSITIVE PAIRS IN THE ROW; – DENOTES AN UNDEFINED PERCENTAGE FOR A ZERO-POSITIVE STORY.

Story	Positive	In	Out	% in
SYS01-US01	3	1	2	33.33
SYS01-US02	1	1	0	100.00
SYS01-US03	3	0	3	0.00
SYS02-US01	5	1	4	20.00
SYS02-US02	5	1	4	20.00
SYS02-US03	2	1	1	50.00
SYS03-US01	1	0	1	0.00
SYS03-US02	5	0	5	0.00
SYS03-US03	2	0	2	0.00
SYS04-US01	0	0	0	–
SYS04-US02	2	0	2	0.00
SYS04-US03	7	0	7	0.00
Pooled	36	5	31	13.89

C. ADR-Generation Completion and Provenance

The frozen generation outputs contain 36 unique ADRs for the 36 LLM-positive story–pattern pairs. The output pair set equals the selected-pair set: every selected pair has exactly one final ADR, with no missing or additional pair. The approved `SYS01-US01`–Pattern `001` pilot is included in this total, not counted again as a separate observation.

All 36 records contain nonempty string-valued title, context, decision, and consequences fields and the fixed status `Proposed`. The offline validation checks exact field membership, pair identifiers, source and rendered-prompt hashes, model configuration, response metadata, and stored strict-output validation flags. Recompiling the frozen records reproduces both the ADR-field CSV and the ADR-enriched selected-pair CSV byte-for-byte. The source hash refers to the preserved pre-ADR selection file, rather than its subsequently enriched version.

The execution history retains a *Connection error* for `SYS04-US03`–Pattern `003`, followed by an explicitly authorized retry linked to the original attempt. The failed attempt and retry authorization are execution metadata, not additional ADRs; the final record for that pair is counted once. The connection failure does not establish whether the failed request reached the provider, so the completed-record count is not a verified total of all API dispatches.

These results establish artifact availability, field completeness, and traceable one-to-one generation for the selected pairs. Nonempty fields, schema conformity, and reproducible compilation do not demonstrate that the decisions are correct, sufficiently grounded, useful, or accepted by architects. `Proposed` is an application-assigned status, not an expert endorsement. Quality judgments are reserved for the separate expert assessment; no such finding is asserted here.

TABLE VII
EXPERT SAMPLE AND RESPONSE-ACCOUNTING SHELL. ALL HUMAN RESULTS ARE UNRESOLVED (TBD), NOT OBSERVATIONS.

Intended result	Unresolved value
Confirmed distinct eligible participants	**TBD**
Submission dispositions and route denominators	**TBD**
Experience categories (valid/unknown denominators)	**TBD**
Architecture/AI knowledge categories	**TBD**
Complete selected-system route evaluations	**TBD**
Partial selected-system route evaluations	**TBD**
Unknown profile values	**TBD**
Missing eligible rating opportunities	**TBD**
Invalid eligible ratings	**TBD**
Observed/expected case–story–system coverage	**TBD**

TABLE VIII
RQ3 RATING-SUMMARY SHELL: NO EXPERT OBSERVATIONS. EACH DISTRIBUTION WILL RETAIN ALL ORDERED CATEGORIES UNDER THE APPROVED CASE–STORY–SYSTEM BALANCING; QUARTILES ARE CATEGORY INTERVALS, AND VALID n IS A RAW COUNT, NOT A WEIGHTING DENOMINATOR.

Item	Distribution	Median	Quartiles	Valid n
Q1	**TBD**	**TBD**	**TBD**	**TBD**
Q2	**TBD**	**TBD**	**TBD**	**TBD**
Q3	**TBD**	**TBD**	**TBD**	**TBD**
Q4	**TBD**	**TBD**	**TBD**	**TBD**
Q5	**TBD**	**TBD**	**TBD**	**TBD**

D. RQ3: Expert Assessment (Unresolved)

EXPERT RESULTS UNRESOLVED—TBD. This subsection contains claim-neutral reporting templates, not empirical findings. No consolidated expert-response dataset has been ingested or analyzed for this draft. Prepared cases and form routes are not completed evaluations or confirmed participants.

Sample template: The confirmed distinct eligible participant count is **TBD**; complete and partial selected-system route evaluations are **TBD** and **TBD**, respectively. Submission dispositions and route denominators are **TBD**. Experience and knowledge summaries are **TBD** and **TBD**, with unknown profiles **TBD**. Missing and invalid eligible ratings are **TBD** and **TBD**; observed/expected coverage is **TBD** (Table VII). Blank unselected routes will not be treated as missing eligible ratings.

Rating templates: For Q1 (adequacy), the response distribution is **TBD**; the median is **TBD** (interquartile category interval **TBD**). For Q2 (grounding), these summaries are **TBD**, **TBD**, and **TBD**; for Q3 (consequences), **TBD**, **TBD**, and **TBD**; for Q4 (perceived usefulness), **TBD**, **TBD**, and **TBD**; and for Q5 (pattern applicability), **TBD**, **TBD**, and **TBD**. Table VIII reserves the balanced study summaries and raw valid counts. System-specific distributions and case/item coverage will accompany the verified results under the approved analysis protocol; no composite score or pass/fail threshold is planned.

Agreement and sensitivity templates: Within-system, per-item ordinal Krippendorff α values or non-estimability reasons are **TBD**; confirmed-rater and pairable-case coverage is **TBD**. Agreement remains conditional on verified distinct raters and shared cases, with no pooled cross-system coefficient. The complete-route sensitivity comparison is **TBD**.

Qualitative template: Reconciled themes, including negative and mixed comments, are **TBD**; comment/opportunity denominators and multiple-code accounting are **TBD**. No respondent quotation or theme is supplied before independent coding, reconciliation, and disclosure review.

Conditional interpretation placeholder: Favorable assessments could support perceived adequacy or usefulness within the assessed cases; mixed assessments could reveal differences between items or systems; unfavorable assessments could identify limitations requiring revision. None of these branches is selected. The evidence-supported direction, scope, and interpretation remain **TBD** until consolidation; later Discussion will retain only supported interpretations. Expert ratings alone cannot establish deployment success or correctness across unassessed patterns or systems.

V. DISCUSSION

Partial Task 4.1 draft. The computational answers below use the approved Results; the expert-assessment answer remains unresolved.

A. Answers to the Research Questions

RQ1—Observed applicability classifications. Within the sampled requirements and catalog, the LLM predominantly returned insufficient information: 773 of 840 pairs (92.02%). It identified 36 pairs as applicable (4.29%), spanning 11 of 12 stories; 31 pairs were classified as not applicable (Table V). Thus, the output is a bounded set of story-specific pattern candidates, not a universal recommendation list. The dominant unresolved category limits what can be concluded about catalog applicability from this run: it does not establish that those patterns are unsuitable or that the requirements are defective. Likewise, positive classifications are model judgments, not validated architectural choices. Differences across stories describe this sample; explaining them would require evidence beyond the reported label frequencies.

RQ2—Overlap with the frozen retrieval baseline. The methods' outputs intersect on 5 of the 36 LLM-positive pairs (13.89%), across 5 stories; 31 positive pairs lie outside the same-story final Top-5 (Table VI). This answers a candidate-membership question: the final Top-5 does not contain most pairs labeled applicable by the exhaustive assessment. It does not establish which method makes better architectural selections. The selected-pair set is not an independent reference, and the two protocols differ in input language and catalog representation. Consequently, non-overlap cannot be interpreted as retrieval error, additional validated discoveries, or superiority of exhaustive assessment.

RQ3—Expert assessment pending. **EXPERT RESULTS UNRESOLVED—TBD**. Table VIII is an unresolved reporting shell, not evidence of expert judgments. The expert judgments

remain unknown. The complete, traceable ADR outputs (Section IV-C) establish generation completion, not professional acceptance or semantic quality. Whether experts corroborate, qualify or challenge the recommendations remains undetermined. Revisit this answer and the combined interpretation after Phase 3 verifies ratings, coverage and eligible comments; no outcome is assumed.

B. Interpreting Exhaustive Assessment and Retrieval

The two protocols have different decision boundaries. Exhaustive assessment evaluates each story–pattern pair against the supplied requirements and returns an applicability category with supporting evidence for positive classifications (Section III-D). The retrieval baseline instead ranks catalog documents by their relationship to a story query and retains a fixed-size candidate set (Section III-E). A ranking position is not an applicability judgment: even a story with no LLM-positive pair receives retrieval candidates. Conversely, exhaustive assessment is not constrained to return a fixed number of positives. Their intersection therefore describes agreement in candidate membership, not agreement on a common classification task.

Table VI shows that the frozen final Top-5 omits most LLM-positive pairs. This observation neither proves that retrieval misses appropriate patterns nor validates the additional LLM selections. English retrieval queries, Portuguese assessment inputs and unequal catalog representations prevent attributing the difference solely to the selection mechanism. The current comparison also does not establish why individual candidates rank outside the cutoff. Architectural quality and relevance require evidence independent of either output set; expert assessment of selected pairs alone cannot establish recall across the unassessed alternatives.

Future design hypothesis—not evaluated. Retrieval could serve as a candidate-generation stage before pairwise LLM assessment, potentially reducing the number of pairs sent for assessment. Such filtering would also exclude the current LLM-positive pairs outside the cutoff from consideration; the architectural importance of those exclusions is unknown. No hybrid pipeline was executed, and end-to-end cost savings, latency and quality effects are **not evaluated**, not demonstrated benefits. A controlled follow-up should align input language and catalog fields, vary the candidate budget, account for retrieval/reranking overhead alongside generation costs, and independently assess candidates both inside and outside the shortlist. Until those trade-offs are measured, retrieval-first assessment remains a proposed alternative rather than a validated replacement for exhaustive assessment.

C. Practical Implications for Architects (Partial)

The current evidence supports treating ADR4AI outputs as material for decision review, not autonomous architectural authority. Requirement references and explicit pattern rationales identify claims an architect can inspect; source/prompt provenance and structured ADR drafts preserve what was generated for each selected pair (Sections III-D and IV-C). These features make the reasoning available for scrutiny, but do not establish that references entail the recommendation, that consequences are complete, or that drafts improve practice. Their value as review aids remains a hypothesis pending professional assessment.

Proposed review checkpoints—not a validated workflow. Before generation, confirm permission to process the requirements and whether the supplied story and acceptance criteria capture the necessary architectural context. For each recommendation, check the cited evidence against the requirements, examine the pattern's fit and alternatives, and identify unsupported assumptions or missing constraints. Before accepting an ADR, review its decision and consequences against system-specific quality requirements and implementation constraints; record revisions and explicit architect approval separately from generation completion. External processing and release require the governance confirmations in Section III-I; the artifacts do not establish that these safeguards are operational.

Unsafe uses include treating the application-assigned `Proposed` status as acceptance, nonempty/schema-valid fields as semantic correctness, or an insufficient-information label as proof that a pattern is unsuitable. Neither a retrieved rank nor an LLM-positive classification warrants automatic deployment. The available evidence does not justify replacing architect review, claiming production readiness, or promising reduced decision effort.

TBD—expert-feedback integration pending. After Phase 3, revise these implications using verified item-specific judgments and eligible comments on usefulness, grounding and clarity, retaining reservations and negative cases. Implementation feasibility and effort are not directly measured by Q1–Q5; discuss them only where actual eligible comments or separately documented evidence support doing so. No such feedback is asserted here, and its absence must not be replaced with assumed benefits. Final recommendations remain contingent on the observed coverage, assessed cases and limitations.

D. Research Implications and Future Work

Requirement-grounded, pattern-by-pattern assessment makes the unit of recommendation explicit and preserves an inspectable link between each positive classification and its cited requirement evidence. Exhaustive coverage also retains non-positive outcomes rather than reporting recommendations alone (Table V). This provides an auditable basis for studying selection behavior, not proof of semantic grounding. The predominance of insufficient-information labels motivates testing how architectural context affects classification, but does not identify missing context as the cause of those labels. Catalog scope, prompt rules and model behavior are alternative factors that the current run does not disentangle.

The study connects selection to documentation through a one-to-one story–pattern-to-ADR mapping (Section IV-C). This connection permits examining the proposed decision together with the selection rationale and requirements. However, the stages are dependent: an unsupported selection can lead to a complete but inappropriate ADR. Reproducible compilation

validates artifact lineage, not decision quality, and the generated ADR is not independent confirmation of its underlying classification.

Proposed follow-up experiments—not evaluated. Replication with additional systems and catalogs should vary domain, requirement detail and pattern coverage explicitly rather than imply generalization from this sample. Controlled comparisons of models and prompt variants should keep requirement inputs and catalog versions fixed, retain all classifications and record repeated-run variability. A context-enrichment experiment could compare the same cases before and after adding verified architectural constraints, distinguishing changes in output frequencies from independently assessed improvements.

Independent expert assessment of sampled non-positive pairs, or an all-pattern reference for a bounded subset, could examine applicability beyond the selected cases and support classification-error measures when the reference judgments are sufficiently established. The planned selected-pair questionnaire alone cannot do so. Selection and ADR quality should also be assessed separately to distinguish recommendation limitations from documentation limitations. Cost and latency evaluation should measure end-to-end execution, including failed attempts, retrieval/reranking and human review where applicable, rather than infer savings from completed-output counts. The shortlist experiment proposed in Section V-B would then test resource–quality trade-offs. None of these extensions has been executed or establishes an evaluated contribution of the present study.

VI. Threats to Validity

A. Construct Validity (Partial)

Architectural information in requirements. User stories and acceptance criteria describe required behavior but may omit quality attributes, deployment constraints or architectural context needed to judge a pattern. System purpose supplies context, not sufficient evidence of applicability by itself. The protocol retains an insufficient-information category and requires requirement references for positive classifications (Section III-D). These safeguards avoid forcing every pair into a binary decision, but neither validate the adequacy of the supplied context nor explain the dominant label distribution in Table V. Reference membership checks do not establish semantic entailment. Independent review of the cited evidence is still needed; controlled context enrichment is a future mitigation, not an intervention already shown to improve classification.

Applicability, retrieval overlap and ADR quality. Applicability is operationalized as a model judgment under the supplied prompt and catalog, not independently established architectural truth. Final Top-5 overlap measures same-story candidate membership relative to the LLM-positive set (Section III-E); it does not measure accuracy, precision or recall. Explicit denominators and separate reporting of classification, overlap and ADR completion limit conflation between these constructs. ADR schema conformity and nonempty fields establish structural completeness, not correctness, sufficient grounding or comprehensive trade-offs (Section IV-C). The planned professional assessment addresses separate judgment dimensions, but cannot convert generation completion into a validated quality measure.

Questionnaire coverage. The prepared Q1–Q5 items distinguish decision adequacy, grounding, consequences, perceived record usefulness and pattern applicability (Table III). This separation and the predeclared item-specific analysis avoid treating different dimensions as an unjustified composite score. Nevertheless, explanatory questionnaire wording and preparation checks do not establish instrument validity or consistent respondent interpretation. Perceived usefulness is not observed practice improvement; the items do not directly measure implementation feasibility, decision effort or deployment success. Optional comments may contextualize judgments, but do not replace missing measurements. Confirming the administered wording and case mapping, examining actual coverage and retaining critical comments are planned safeguards, not completed empirical validations.

Evaluation of selected pairs only. The prepared questionnaire presents LLM-positive pairs and their generated ADRs. This permits assessment of those recommendations, not a catalog-wide reference set. Rejected and insufficient-information pairs are outside the prepared evaluation, so the design cannot establish missed applicable patterns, false-negative rates or comparative selection accuracy. Restricting claims to assessed cases is the current mitigation. Independent assessment of non-positive pairs or all patterns for a bounded subset is proposed future work (Section V-D), not evidence supplied by the selected-pair questionnaire.

TBD—administered-instrument and expert-evidence reconciliation pending. After Phase 3, verify actual wording, routing, eligibility and coverage, document deviations from the prepared protocol, and revise these construct limits against the supported findings. No completed instrument validation or expert-based mitigation effect is claimed in this partial draft.

B. Internal Validity (Partial)

Prompt design and model variability. The decision rules, supplied catalog fields and ADR prompt's instructional example may influence classifications and generated decisions. The frozen study uses one recorded model/reasoning configuration for these stages, without a controlled prompt comparison or repeated-run robustness analysis (Sections III-D and III-F). No temperature or seed is supplied. Separate requests exclude prior conversation and generated outputs, limiting conversational carryover, but do not guarantee deterministic responses or statistical independence across shared stories and patterns. Prompt/source hashes and preserved outputs identify the evaluated run; they do not isolate prompt effects or prove that another run would agree. Controlled prompt/model comparisons and repeated executions remain future checks, not safeguards already shown to remove these threats.

Translation and authorized corrections. Portuguese presentation may alter how evaluators perceive the selected patterns

and ADRs. The preparation process retains the original requirements literally and records source text, translation candidates, final presentation and authorized field-level corrections separately (Section III-G). Exact mapping and preservation checks make changes auditable, but do not independently establish translation equivalence or exclude presentation bias. Linguistic corrections are not architectural corrections; they must not be interpreted as improving the generated decisions. Actual administered text must be reconciled with these frozen versions, with deviations reported rather than attributed to the original outputs.

Execution and retry effects. Strict structured outputs, identifier/reference checks, retained completed records and provenance audits reduce malformed-output and join errors, not unsupported reasoning. Applicability attempt reservations and its bounded retry policy differ from ADR generation's stop-on-failure and explicit manual retry authorization. These policies can affect which response becomes the retained outcome; failures are not applicability labels or extra observations. The preserved pilot boundaries and attempt history make those choices inspectable. The authorized ADR connection-error retry is linked to its prior attempt and yields one final record (Section IV-C); failed dispatch remains unknown. Complete pair coverage limits missing-output ambiguity but does not establish retry-neutral results, semantic correctness or an exact global API-call count.

Evaluator learning, fatigue and analysis choices. Repeated cases and unequal prepared route lengths may induce learning, anchoring or fatigue, changing judgments across a session. System-specific routing limits the prepared workload to the selected system, but fixed presentation order does not counterbalance order effects. Static routing checks do not establish actual live behavior, completion patterns or the absence of fatigue. The approved pre-inspection analysis protocol specifies eligibility, missingness, aggregation, sensitivity and comment coding, reducing discretion in later analysis; it is not preregistration before collection and does not control administration. Any amendments and prior exposure to responses must remain documented.

TBD—expert-administration and analysis reconciliation pending. After Phase 3, confirm actual routing, presentation order, administration conditions, completion/coverage and protocol deviations. Report only supported evidence of evaluator effects and executed safeguards; do not infer fatigue from route size or claim that planned analysis procedures have already been applied.

C. External Validity (Partial)

Industrial sample and requirement scope. The empirical evaluation relies on 4 AI-enabled systems (SYS01–SYS04), comprising 12 user stories (3 per system) and 35 validated acceptance criteria (Section III-B). All four systems originate from a single public-sector organization in Brazil. Consequently, requirements reflect specific organizational practices, enterprise workflows, regulatory constraints, and public-administration governance. Architectural tradeoffs prevalent in other domains—such as high-frequency financial transactions, ultra-low-latency edge devices, or safety-critical autonomous systems—are not represented in this corpus. While the four systems span distinct functional capabilities (e.g., text summarization, legal analysis, conversational assistance, and automated triage), and the evaluation covers an exhaustive census of all 840 story-pattern pairs without subsampling, the findings cannot be generalized to all AI-enabled systems or different organizational cultures without multi-organization replication.

Pattern catalog boundaries and coverage. The study uses a curated catalog of 70 architectural patterns for AI-enabled systems drawn from literature and the SWE4AI repository [4], [5]. These patterns emphasize data management, model training, deployment, and monitoring. The catalog does not cover general software architectural styles (e.g., event-driven architectures, microservices, or layered designs) or traditional quality-attribute tactics (e.g., hardware fault tolerance or network redundancy), nor does it incorporate recent generative AI design motifs emerging after the catalog freeze. The structured catalog schema ensures uniform field representation across all 70 patterns during prompting, but pattern selection frequencies, insufficient-information rates, and generated ADR structures may differ under alternative, expanded, or differently structured architectural taxonomies.

Language, translation and domain context. The original industrial requirements and acceptance criteria were formulated in Brazilian Portuguese, whereas the pattern catalog definitions and the primary prompt templates are in English (Section III-G). Model-based pattern assessment therefore requires multilingual semantic alignment between Portuguese operational needs and English architectural specifications. While the study preserves original Portuguese requirements verbatim and audits translation artifacts to avoid semantic distortion, domain-specific public-administration vocabulary and linguistic nuances may influence prompt comprehension. The observed model behaviors and ADR generation qualities may not transfer identically to monolingual English requirements or other linguistic and regional contexts.

Model and pipeline specificity. The selection and generation pipelines rely exclusively on a single commercial frontier model (`gpt-4o-2024-08-06`), structured JSON generation, and a specific embedding model (`text-embedding-3-small`) for the RAG baseline (Sections III-D and III-E). The resulting applicability distributions, high proportion of insufficient-information outcomes (773 of 840), and specific ADR trade-off formulations reflect the reasoning capacity, instruction-following behavior, and pretraining distribution of this specific model snapshot. These results do not generalize to alternative commercial models, open-weight architectures (e.g., Llama or Mistral variants), different prompt strategies (such as multi-step chain-of-thought or multi-agent debate), or other retrieval configurations without direct comparative evaluation.

TBD—Evaluator sample and practitioner representativeness pending Phase 3 expert consolidation. When human evaluation data is consolidated in Phase 3, this subsection must

be updated with the empirical demographics of the participant cohort, including professional tenure, software architecture experience, evaluated-system familiarity, and the balance between internal system engineers and independent architects. The generalizability of expert ratings and qualitative feedback must be explicitly bounded by the rater sample composition, avoiding unsupported extrapolations to the wider software engineering community.

D. Conclusion and Reproducibility Validity (Partial)

Descriptive analysis and ordinal limits. The quantitative findings in this study are strictly descriptive and exploratory. Model classifications and retrieval overlaps are reported as exact counts and percentages over clear census denominators (Sections IV-A and IV-B). Because the study conducts an exhaustive census of all 840 story-pattern pairs across the four participating systems without random sampling, inferential statistical tests, p-values, and confidence intervals are not computed for the computational outcomes. Similarly, the predeclared expert evaluation protocol specifies ordinal medians and quartile intervals rather than parametric means or composite scores, preventing unwarranted interval-scale assumptions.

Hierarchical aggregation and small cell counts. The multilevel structure of the data—spanning 840 assessments across 12 stories and 4 systems—involves inherent dependencies. The computational results present transparent, unpooled story-level and system-level breakdowns alongside overall totals (Tables V and VI), avoiding Simpson's paradox or distorted conclusions driven by high-yield stories. However, for rare outcomes (such as the 5 overlapping pairs in RQ2 or the single story SYS03-US03 with zero positive classifications), small sample sizes limit fine-grained sub-group inferences.

Third-party model evolution and unpinned service revisions. The experimental pipeline depends on commercial black-box APIs hosted by OpenAI (specifically `gpt-4o-2024-08-06` for inference and `text-embedding-3-small` for dense retrieval). While dated model snapshots were targeted during execution, external API providers may apply backend infrastructure updates, unversioned floating-point optimizations, or deprecate model snapshots over time. Furthermore, unpinned embedding revisions could alter nearest-neighbor retrieval rankings. Exact bit-level replication across external cloud endpoints cannot be guaranteed independently of the service provider's lifecycle.

Artifact preservation and deterministic compilation. To maximize reproducibility within the boundaries of closed-model APIs, all intermediate prompt inputs, system instructions, raw JSONL model outputs, retrieval rankings, and final Markdown ADRs are immutably preserved with cryptographic SHA-256 hashes and timestamped audit logs (Section III-I). All manuscript numerical values, summary tables, and protocol parameters are programmatically compiled from verified offline data records into isolated LaTeX macro definitions, ensuring that the reported empirical results are fully deterministic and auditable from the preserved experimental outputs.

TBD—Statistical conclusion validity and inter-rater agreement pending Phase 3 expert consolidation. When human evaluation data is consolidated in Phase 3, this subsection must be updated with the empirical sample sizes, missingness rates, and rater coverage across evaluated cases. Within-system and item-level inter-rater agreement (ordinal Krippendorff's α) must be calculated only where distinct raters evaluate shared cases with positive expected disagreement; otherwise, non-estimability and its exact mathematical cause must be formally documented. No inter-rater reliability, statistical consensus, or significance claims may be asserted prior to verified Phase 3 execution.

References

[1] A. Jansen and J. Bosch, "Software architecture as a set of architectural design decisions," in *5th Working IEEE/IFIP Conference on Software Architecture (WICSA)*. IEEE, 2005, pp. 109–120.

[2] P. Kruchten, P. Lago, and H. van Vliet, "Building up and reasoning about architectural knowledge," in *Quality of Software Architectures (QoSA)*, ser. Lecture Notes in Computer Science, vol. 4214. Springer, 2006, pp. 43–58.

[3] H. Washizaki, F. Khomh, Y.-G. Guéhéneuc, H. Takeuchi, and S. Katsumata, "Software engineering design patterns for machine learning applications," in *2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC)*. IEEE, 2019, pp. 392–399.

[4] L. Heiland, M. Hauser, and J. Bogner, "Design patterns for AI-based systems: A multivocal literature review and pattern repository," in *2023 IEEE/ACM 2nd International Conference on AI Engineering – Software Engineering for AI (CAIN)*. IEEE, 2023, pp. 184–196.

[5] SWE4AI, "Design patterns for AI-based systems: A web-based repository," [Online]. Available: https://swe4ai.github.io/ai-patterns/, 2023, repository snapshot 9ff3e61; accessed October 12, 2026.

[6] M. Nygard, "Documenting architecture decisions," [Online]. Available: https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions, 2011, November 15; accessed October 12, 2026.

[7] H. Cervantes, Y. Cai, and R. Kazman, "Large language models as assistants in software architecture design," *IEEE Software*, vol. 43, no. 4, pp. 87–95, 2026.

[8] R. Dhar, K. Vaidhyanathan, and V. Varma, "Can LLMs generate architectural design decisions? An exploratory empirical study," in *2024 IEEE 21st International Conference on Software Architecture (ICSA)*. IEEE, 2024, pp. 79–89.

[9] M. Soliman and J. Keim, "Do large language models contain software architectural knowledge? An exploratory case study with GPT," in *2025 IEEE 22nd International Conference on Software Architecture (ICSA)*. IEEE, 2025, pp. 13–24.

[10] F. R. de Oliveira, F. V. D. Souza, A. L. B. D. P. Barros, and P. H. Maia, "LLMs as design partners for AI-based system patterns: An empirical evaluation," in *Proceedings of the IEEE/ACM 5th International Conference on AI Engineering – Software Engineering for AI (CAIN '26)*. ACM, 2026, pp. 12–22.

[11] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[12] H. Cervantes, R. Kazman, and Y. Cai, "An LLM-assisted approach to designing software architectures using ADD," *arXiv preprint arXiv:2504.04334*, 2025.

[13] X. Zhou, R. Li, P. Liang, B. Zhang, M. Shahin, Z. Li, and C. Yang, "Using LLMs in generating design rationale for software architecture decisions," *ACM Transactions on Software Engineering and Methodology*, 2026, in press, arXiv:2504.20781.

[14] Z. R. Jánki and V. Bilicki, "Rule-based architectural design pattern recognition with GPT models," *Electronics*, vol. 12, no. 15, p. 3364, 2023.